

ARCTIS Project Specification

- [ARCTIS — Project Specification](#)
 - [1. Product Vision](#)
 - [2. Architecture Overview](#)
 - [3. AI Routing Architecture \(Implemented\)](#)
 - [4. Transfer/Swap/Bridge AI Orchestration \(Implemented\)](#)
 - [5. Credits System \(Implemented\)](#)
 - [6. Agent Architecture \(Implemented, pre-existing + this cycle's fixes\)](#)
 - [7. Database Design](#)
 - [8. API Overview](#)
 - [9. Activity Center & History \(Implemented\)](#)
 - [10. Command Palette \(Implemented\)](#)
 - [11. AI Memory \(Not Implemented — Future Roadmap\)](#)
 - [12. UI/UX Design Principles \(Implemented — theme system + visual pass\)](#)
 - [13. Mobile Design](#)
 - [14. Security Model](#)
 - [15. Environment Variables \(names only\)](#)
 - [16. Installation / Build / Deployment](#)
 - [17. Testing Guide](#)
 - [18. Feature List \(Implemented, this cycle only — see CHANGELOG.md \[1.1.0\] for full detail\)](#)
 - [19. Known Limitations](#)
 - [20a. Developer Tooling — Keeping Circle Agent Stack Updated](#)
 - [20b. Future Roadmap \(Explicitly Not Implemented\)](#)

ARCTIS — Project Specification

Status: Phase 23 complete (Firestore/AI-routing rebuild, light theme foundation, page-by-page premium redesign, accessibility/security audit). This document describes the codebase as implemented, verified via static analysis (see [TEST_REPORT.md](#) and [BUILD_REPORT.md](#)). Sections marked **Future Roadmap** are explicitly not implemented — do not read them as current functionality.

1. Product Vision

ARCTIS is an AI Operating System built on Arc Testnet, combining four pillars into one platform:

- **Knowledge OS** — sessions, saved prompts, workspace domains
- **AI OS** — 12 AI personas over a single, provider-agnostic backend the user only ever knows as “ARCTIS AI”
- **Stablecoin OS** — Transfer, Swap (real OTC), Bridge (CCTP V2), each with a full on-chain proof chain
- **Economic Agent OS** — 7 agent types with memory, monthly budgets, an independent evaluator, and a mandatory human approval gate before any agent action executes

The product is designed to feel like a single coherent AI platform, not a thin wrapper around a third-party API — backend routing, providers, and model identities are an implementation detail, never a user-facing concept.

2. Architecture Overview

```
Browser (Next.js client components)
|   - no direct Firestore access anywhere
|   - no financial transaction ever signed by anything but the user's own wallet
▼
Next.js API Routes (server-only)
```

```
|
├─ Firebase Admin SDK → Firestore (rules deny all direct client access)
├─ AI Router → Model Registry + Health Tracker → OpenRouter
├─ Arc Testnet (via user's wallet signature, never server-signed)
```

Full detail: `SYSTEM_ARCHITECTURE.md`.

3. AI Routing Architecture (Implemented)

Files: `src/lib/ai/router/index.ts`, `src/lib/ai/registry/openrouterModels.ts`, `src/lib/ai/registry/health.ts`, `src/lib/ai/providers/openrouter.ts`

- The model registry discovers free, chat-capable OpenRouter models live via `GET https://openrouter.ai/api/v1/models`, filtering for zero-priced / `:free`-suffixed models, caching for 4 hours, and falling back to the last-known-good list (or a single hardcoded safety net, `openrouter/free`) if OpenRouter is unreachable.
- The health tracker keeps an in-memory score per model — consecutive failures, a cooldown window (escalating on repeat failures, fixed 30s on a 429 rate-limit), and a rolling average latency.
- `routeAIRequest` (non-streaming) and `routeAIStream` (streaming) both walk the health-ranked candidate list and fail over automatically. Streaming only fails over if nothing has been shown to the user yet, to avoid mixing partial output from two different models mid-reply.
- The client never supplies or sees a model ID. `/api/ai/chat` and `/api/ai/copilot` accept only `mode` (persona) and `messages`; the router decides everything else.
- Twelve personas (`MODE_PROMPTS`, keyed by `AIMode`) define behavior via system prompt only — model selection is uniform across all of them, since capability differentiation by model isn't reliable for a dynamically-discovered free-model pool.
- **In-memory, single-process design.** Health state and the registry cache reset on server restart and do not share state across multiple server instances. Documented as a deliberate trade-off for the current deployment; the function signatures are written so a shared store (Firestore/Redis) could replace the in-memory map without touching call sites.

Not implemented (Future Roadmap)

- Cross-instance shared health state
- Per-persona model capability tagging (e.g. routing “research” mode preferentially to a stronger reasoning model)
- Additional provider adapters (Groq, Gemini, Azure, Ollama, local models) — the adapter boundary (`src/lib/ai/providers/`) is structured to allow this, but only the OpenRouter adapter exists today

4. Transfer/Swap/Bridge AI Orchestration (Implemented)

Files: `src/lib/ai/intent/parser.ts`, `src/app/api/ai/chat/route.ts` (intent-detection block), `ActionProposalCard` in `src/app/ai/page.tsx`, `pendingAction` in `src/lib/store/index.ts`

A message like “*bridge 5 USDC*” is matched by a **deterministic regex parser**, not an LLM call — extracting an amount, address, or token pair via an LLM risks a subtly wrong number or address, which is unacceptable for a financial action. If matched:

1. No AI model is called, no credits are charged.
2. The chat route returns an `actionProposal` payload instead of a normal reply.
3. The frontend renders a confirmation card (amount, tokens, recipient) with **Confirm & Continue / Dismiss**.
4. Confirming sets `pendingAction` in the shared store and navigates to the existing `/transfer`, `/swap`, or `/bridge` page.
5. That page's own `useEffect` reads `pendingAction`, pre-fills its form, clears the pending action, and the user completes the **same existing wallet-signing flow** as if they'd navigated there directly — no new execution logic exists anywhere in this feature.
6. Once the user completes the transaction on the existing page, it's recorded exactly as it always was (`saveTransaction`, `createSwapRecord`, `createBridgePending`), which is what already feeds `/api/activity` → both Activity Center and History.

Explicit safety property: at no point does any server-side code sign, submit, or simulate a transaction. Wallet signing is 100% client-side, on the pre-existing pages, unchanged.

Not implemented (Future Roadmap)

- Multi-step or ambiguous requests (“send some USDC to my friend” without an amount) — the parser requires an explicit amount, token, and (for transfer) a full address; anything else falls through to a normal AI reply
- Treasury natural-language queries/actions (mentioned in early planning, not built — Treasury remains observer-only per the existing architecture lock, and no NL interface was added for it)

5. Credits System (Implemented)

- **Flat rate:** 1 credit per 1,000 tokens, regardless of which backend model actually served the request (`CREDITS_PER_1K_TOKENS` in `src/lib/memberships/plans.ts`). There is no per-model cost table anymore — one existed before this cycle and was removed because it was both inaccurate (never actually wired into the deduction math, which was always flat) and a model-name leak (rendered directly in the Credits page and onboarding flow).
- ARCTIS Copilot never deducts credits, regardless of message volume.
- Ledger (`credit_ledger` collection) records every purchase/deduction/bonus/refund with a before/after balance snapshot. The description text is always branded (“ARCTIS AI — Research mode”), never a raw model name; the actual model is stored separately in an `aiModel` field that is never rendered in any UI.
- Balance (`credit_balances` collection) tracks lifetime `total` and `used` ; `remaining` is computed on read.

Not implemented (Future Roadmap)

- Tiered/variable pricing by task complexity (a “persona-tier” cost model was discussed but the flat rate was kept, matching “do not redesign billing”)

6. Agent Architecture (Implemented, pre-existing + this cycle’s fixes)

Seven required agent types plus template-based custom agents. Every agent action follows **Propose → Review → Approve → Execute** — enforced in `src/lib/agents/executor.ts` itself (`proposeAgent` , `approveProposal` , `rejectProposal`), not just hidden behind UI. An Independent Evaluator (`src/lib/agents/evaluator.ts`) makes a structurally separate inference call with no access to the generator’s prompt or memory, for adversarial review, with one bounded revision on failure.

This cycle: removed a membership-tier model-access gate from `executor.ts` (contradicted the free-models-only design, and was actively blocking agent execution for any non-Pro-tier wallet); removed a hardcoded paid model (`anthropic/claude-3.5-haiku`) from the evaluator; fixed Firestore writes (Admin SDK) so `createAgent` actually persists instead of showing a “Saved” toast over a silently-failed write; removed the “AI Model” dropdown from the Create Agent form (model selection is automatic, same as chat).

Not implemented (Future Roadmap)

- Agents autonomously triggering Transfer/Swap/Bridge (the orchestration feature in §4 is chat-initiated and always requires confirmation; agents themselves do not yet call these services)

7. Database Design

Full collection-by-collection reference: `DATABASE.md` . Summary: 17 Firestore collections, all read/written exclusively via the Firebase Admin SDK from server-side API routes. `firestore.rules` denies all direct client access except a public, read-only `passports/{username}` lookup. 14 composite indexes cover every query added across this project’s lifetime (`firestore.indexes.json`).

8. API Overview

Full reference: `API_REFERENCE.md` . Grouped by pillar: Stablecoin OS (Credits, Membership, Swap, Bridge), Identity (Passport), Economic Agent OS (propose/approve/execute/stream/reports), AI OS (`chat` , `copilot` , `sessions` , `prompts`), Platform (`activity`).

9. Activity Center & History (Implemented)

Both read from **one shared aggregation endpoint**, `GET /api/activity` — no duplicate backend logic. Activity Center is a live, filterable card feed; History is a denser, paginated table with richer export options (CSV/JSON/Excel/PDF/TXT). Neither ever renders a backend model/provider name — the aggregation route's internal `meta` field is stripped of model identity before being sent to the client.

This cycle: added Swap/Bridge event types to Activity Center's filters; rebuilt History entirely (it previously read only from local, unsynced browser state and appeared empty on any new device).

10. Command Palette (Implemented)

`src/components/ui/CommandPalette.tsx` , mounted globally in the dashboard shell. `⌘K` / `Ctrl+K` or the header's "Search" button opens a fuzzy-filterable list of every top-level page, grouped by pillar (Overview, AI OS, Knowledge OS, Stablecoin OS, Finance, Platform). Arrow-key navigation, Enter to go, Escape to close. State lives in the shared Zustand store (`commandPaletteOpen`), not local component state, so any part of the app can open it.

Not implemented (Future Roadmap)

- Executing actions directly from the palette (e.g. "Create Agent" opening the create form pre-focused, rather than just navigating to `/agents`)
 - Recent/frequently-used action ranking
-

11. AI Memory (Not Implemented — Future Roadmap)

Session persistence (`ai_sessions`) and the saved-prompt library (`saved_prompts`) already exist and predate this cycle — these give continuity within a conversation and a personal prompt library, respectively. A distinct "AI Memory" layer (remembered preferences, favorite tools, cross-session personalization beyond what Copilot's dynamic context already builds) was scoped in early planning for this cycle but was **not built** — deprioritized in favor of the Firestore fix, AI routing rewrite, and Transfer/Swap/Bridge orchestration, which were higher-impact within the 2-3 day window. Do not represent this as implemented.

12. UI/UX Design Principles (Implemented — theme system + visual pass)

Light is the default theme; dark mode is fully supported, not removed. The surface color scale (`src/app/globals.css` , `tailwind.config.ts`) was converted from static hex values to CSS custom properties — `:root` defines a soft, warm off-white palette (Apple/Linear/Notion-inspired, not stark clinical white) and `.dark` overrides every one of those variables back to the original dark palette this project shipped with, preserved exactly. Because every page already used the same `surface-*` token names consistently, this single foundation change re-themes the entire app without needing to restructure any page's JSX.

Theme system (`src/lib/theme/ThemeProvider.tsx`): a small, dependency-free provider — no `next-themes` or similar package added. Toggles a `.dark` class on `<html>` , persists the choice to `localStorage` , and an inline script in `src/app/layout.tsx` 's `<head>` applies the stored preference before React hydrates, avoiding a flash of the wrong theme on load. A toggle lives in Settings (Light/Dark segmented control) for a considered choice, plus a quick-access control in the dashboard header.

Systemic contrast fixes, applied via script across 28 files rather than by hand (mechanical transforms are lower-risk than 400+ individual edits at this scale): - 132 instances of `border-white/[opacity]` / `bg-white/[opacity]` (subtle borders and hover highlights, originally tuned only for a dark background) → each became a `black` -based light-mode default with the original `white` variant preserved behind `dark:` . - 265 instances of `text-{color}-400` (status badges,

icons — a shade with poor contrast against a light background) → light mode uses the same color's `-600` shade for adequate contrast; `dark:text-{color}-400` preserves the original. - **A bug in the first pass, found and fixed:** roughly 35 of the border/background substitutions above had a `hover:` / `focus:` / `group-hover:` modifier on the light (black) side but not on the corresponding `dark:` (white) side — meaning in dark mode those elements would show the highlight *permanently* rather than only on hover. A second script pass specifically detected and corrected every instance of this pattern.

Components made theme-reactive (previously hardcoded to dark-only literal values, since Recharts and RainbowKit need actual color strings, not Tailwind classes): the RainbowKit wallet-connect modal (`WalletProviders.tsx`), toast notifications (`Providers.tsx`), and the Treasury/Analytics chart tooltips and grid lines (both pull their color from `useTheme()` at render time).

Other fixes found during the pass: - `.page-container` was referenced by 5 pages but was never actually defined anywhere — a silent no-op className. Now defined with sensible default spacing. - No `@layer base` typography rules existed at all — headings had no consistent tracking/weight baseline beyond whatever each page happened to apply inline. Added one. - The landing page's decorative grid-line background was white-tinted only, making it invisible against the new light default background. Now theme-aware. - Existing design tokens (`.glass-card`, `.btn-primary` / `.btn-ghost`, the shimmer `.skeleton` class, Framer Motion easing curves) were already a reasonably strong foundation from earlier cycles — this pass reinforced and extended them rather than replacing the system. - `EmptyState` and `Skeleton` / `SkeletonRow` / `SkeletonList` / `SkeletonCard` (`src/components/ui/`) — shared primitives from an earlier cycle, still applied to History, Activity, Credits, Dashboard, Agents.

Verification: every one of the transforms above was checked with `tsc --noEmit` before and after, diffed by error code — the counts moved only in proportion to the small amount of new JSX added (theme toggle, ThemeProvider), and the three error categories worth manually tracing were byte-for-byte unchanged. See `BUILD_REPORT.md` §“UI Overhaul Verification” for the full table.

Page-by-page redesign (Phase 22) and accessibility/security audit (Phase 23): beyond the theme foundation, all 13 priority pages (Dashboard, AI Workspace, Copilot, Agents, Transfer, Swap, Bridge, History, Activity Center, Credits, Passport, Settings, Onboarding) were individually reviewed — not just re-themed. This found and fixed several real bugs a purely visual pass wouldn't catch: a completely unreachable Agent-archive feature, an Export menu that only opened on hover (never worked on touch devices), a Copilot sidebar that broke the layout on phones, two more undefined CSS classes (`.btn-secondary`), and markdown rendering that was never wired up despite the packages being installed. A dedicated accessibility pass added `focus-visible` styling to every button variant (previously only inputs had one), `aria-label` coverage, and an accessible name on the shared `Toggle` component. Full detail in `CHANGELOG.md` [1.2.0] and [1.3.0], and `BUILD_REPORT.md`.

Honest limitation: no page was visually rendered in a real browser during this cycle (no `node_modules`, no dev server, no network in this sandbox). Every claim above is verified at the CSS/logic level, not by looking at a screenshot. The single most important manual check after this ZIP is applied is simply opening the app in both Light and Dark mode and looking at it — see `TEST_REPORT.md`.

Not implemented (Future Roadmap)

- A full page-by-page bespoke redesign — this pass is a systematic foundation fix plus targeted improvements, not a from-scratch visual rebuild of every component
- Dashboard “Quick Actions” / “System Health” sections (mentioned in early planning; still out of scope — a content addition, not a visual polish item)
- System-preference-based theme default (`prefers-color-scheme`) — the app defaults to light regardless of OS setting; only an explicit user choice (via the toggle) switches it

13. Mobile Design

Transfer/Swap/Bridge use a single-column `max-w-lg` layout that is inherently mobile-safe without needing many breakpoints. Not every page was individually re-audited at every breakpoint this cycle — see `TEST_REPORT.md` for the explicit scope of what was and wasn't reviewed.

14. Security Model

See `SYSTEM_ARCHITECTURE.md` §“Security Model” and `FINAL_PROJECT_STATUS.md` §“Security Posture Detail” for the full,

current, honest account — including which routes have full cryptographic wallet-signature verification versus ownership-only checks, and the Firestore Admin SDK / rules-lockdown design introduced this cycle.

15. Environment Variables (names only)

See `.env.example` in the repository root for the authoritative list. Categories: Firebase client config (`NEXT_PUBLIC_FIREBASE_*`), Firebase Admin SDK (`FIREBASE_PROJECT_ID` , `FIREBASE_CLIENT_EMAIL` , `FIREBASE_PRIVATE_KEY` — added this cycle), OpenRouter (`OPENROUTER_API_KEY`), WalletConnect (`NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID`), app URL (`NEXT_PUBLIC_APP_URL`). No values are reproduced here or anywhere in this documentation set.

16. Installation / Build / Deployment

See `README.md` (Quick Start) and `DEPLOYMENT.md` . This cycle added two new dependencies not yet installed in any real environment: `firebase-admin` , `server-only` . Firestore rules/indexes corrected this cycle are not yet deployed to any live project — see `BUILD_REPORT.md` §“Remaining Operational Tasks.”

17. Testing Guide

See `TEST_REPORT.md` for exactly what was and wasn't verified this cycle, and which user journeys require a manual pass in a real environment before this cycle is considered production-verified.

18. Feature List (Implemented, this cycle only — see CHANGELOG.md [1.1.0] for full detail)

- Firebase Admin SDK migration across 14 files; Firestore rules locked down
 - Dynamic free-model registry + health-based AI routing (streaming and non-streaming)
 - ARCTIS AI branding — model/provider identity hidden from users everywhere
 - Transfer/Swap/Bridge AI orchestration (confirm-then-handoff)
 - Command Palette
 - History page rebuilt on the shared activity feed
 - Activity Center — Swap/Bridge event types added
 - Shared EmptyState/Skeleton components, applied across 5 pages
 - Various bug fixes: stream-error swallowing, Credits loading-state flash, silent fetch-error swallowing, one hardcoded paid model, one self-introduced regression caught by `tsc --noEmit`
-

19. Known Limitations

See `FINAL_PROJECT_STATUS.md` §“Known Gaps” — the authoritative, current list. Highlights specific to this cycle: new dependencies not yet installed anywhere real; Firestore rules/indexes not yet deployed; no user journey in this cycle has been exercised in an actual browser against a running server.

20a. Developer Tooling — Keeping Circle Agent Stack Updated

Developer documentation only — describes an external CLI workflow, not application logic. Nothing below is executed by ARCTIS itself.

ARCTIS is built on Arc Testnet, part of Circle's infrastructure — so it's worth keeping the Circle CLI and Circle Skills current as Circle ships updates to the Agent Stack.

Requirement: Circle CLI v0.0.6 or newer.

Update the CLI:

```
circle update
```

Update Circle Skills for Claude Code:

```
circle skill update --tool claude-code
```

Reference: developers.circle.com/agent-stack

Run both commands periodically (e.g. before a hackathon demo or a new development cycle) so ARCTIS's tooling stays compatible with the latest Circle Agent Stack patterns and products. Neither command is part of ARCTIS's build or runtime — this is a local developer-environment maintenance step only.

20b. Future Roadmap (Explicitly Not Implemented)

- AI Memory beyond existing session/prompt persistence (§11)
- Additional AI provider adapters beyond OpenRouter (§3)
- Per-persona model capability tagging (§3)
- Agents autonomously initiating Transfer/Swap/Bridge without a chat-confirmed handoff (§6)
- Command Palette action-execution (not just navigation) (§10)
- Full page-by-page UI redesign beyond this cycle's consistency pass (§12)
- Dashboard Quick Actions / System Health sections (§12)
- Full SIWE/cryptographic signature enforcement on every mutating route (pre-existing gap, tracked in `FINAL_PROJECT_STATUS.md`)
- QR codes and avatar upload on Passport profiles (pre-existing gap, blocked on package installation)
- Reference/Agent/Passport Registry smart contracts (pre-existing gap, blocked on Foundry access)